

Платформа Chronos

Централизованное управление Docker Compose на нескольких узлах

Глеб Михалев

Выпускная квалификационная работа

2026

- Несколько машин с **Docker**, стек описан в **Compose** — привычно и достаточно для многих сценариев.
- Полноценный **Kubernetes** часто избыточен: control plane, экспертиза, стоимость владения.
- **Панели** (Portainer и др.) удобны для UI, но слабее выражена связка «код сборки / CI → та же модель, что на сервере».

Нужен управляющий контур вокруг Compose без обязательного K8s.

Chronos — программный комплекс для централизованного управления Compose-проектами на наборе агентов:

- единый **Master** (метаданные, политики, UI);
- **Agent** на каждом узле с Docker (исполнение `docker compose`);
- библиотека **Core** для описания и публикации из .NET и CI.

Принцип

Docker и Compose не заменяются: Chronos координирует выкладку, наблюдаемость, ТСП и бэкапы.

Три опорных компонента

Chronos.Core Fluent API и YAML, валидация, пакеты деплоя, клиент публикации, тесты и jobs ([Test], [Job]).

Chronos.Agent ASP.NET Core, docker.sock, каталог проектов, логи, снимки томов, регистрация на Master.

Chronos.Master + Web PostgreSQL, REST, прокси UI к агентам, политики бэкапа, реестр TCP для HAProxy.

Поток публикации (упрощённо)

- 1 Разработчик или CI: модель в коде (Core) или готовый compose-файл.
- 2 PublishAsync (и родственные): архив манифеста и артефактов → HTTP на нужный Agent.
- 3 Agent сохраняет файлы и выполняет жизненный цикл через docker compose.
- 4 Master хранит размещение, аудит; UI опрашивает Master, тяжёлые данные (логи) — прокси к агенту.

Резервное копирование томов

- **Политика** на Master: проект, шаблон тома, MaxCopies, S3-префикс.
- Фоновый VolumeReplicationHostedService: свободное место на агенте, листинг бакета, prune лишних копий.
- Agent: снимок тома (tar/gzip) → загрузка в S3-совместимое хранилище (MinIO и т. д.).

ТСР снаружи: HAProxy

- Внешние клиенты подключаются к **listen-порту HAProxy**; backend — адрес сервиса на хосте агента (например `host.docker.internal:PORT`).
- Master ведёт JSON-реестр маршрутов и генерирует фрагмент `chronos-tcp.cfg` (UTF-8 без BOM).
- На стенде: смена файла и сигнал reload HAProxy без ручного редактирования конфигов оператором.

- **Chronos.Master.Web** (React): проекты, агенты, маршруты TCP, песочница YAML/Fluent API и др.
- Контрольный стенд: docker-compose (PostgreSQL, Master, Agent, HAProxy, Loki, Grafana).
- Наблюдаемость: централизованные логи (Loki/Grafana) в связке с агентом и мастером.

- Реализованы и согласованы Core, Agent, Master и UI под описанный сценарий.
- Подтверждена работоспособность на локальном compose-стенде: публикация, ТСР, политики бэкапа, наблюдаемость.
- **Цель ВКР достигнута**; ограничения MVP (один Master, нагрузочные тесты) зафиксированы как направления развития.

Направления развития

- Отказоустойчивость и масштабирование Master.
- Модель прав доступа и ролей.
- Автоматизация подсказок по портам из состояния compose.
- Расширенные нагрузочные испытания.

Спасибо за внимание!

Вопросы?